

Interpretación de CNNs

Clase práctica 22

Motivación

- ¿Cómo podemos visualizar qué pasa en los **filtros convolucionales**?
- ¿Cómo podemos identificar a qué prestan atención las CNNs mediante **saliency maps** con PyTorch?
- ¿Cómo podemos usar **Grad-CAM** con PyTorch y en qué se diferencia de los saliency maps?

Organización

Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:45	05'
Visualización de filtros	08:05	09:50	30'
Saliency maps	08:35	10:00	30'
Grad-CAM	09:05	10:10	30'

Visualización de filtros convolucionales

Los **filtros convolucionales** nos permiten “**ver**” (un poco) lo que pasa en el interior de las redes convolucionales. Esto puede darnos una intuición de por qué una red convolucional funciona o no funciona.

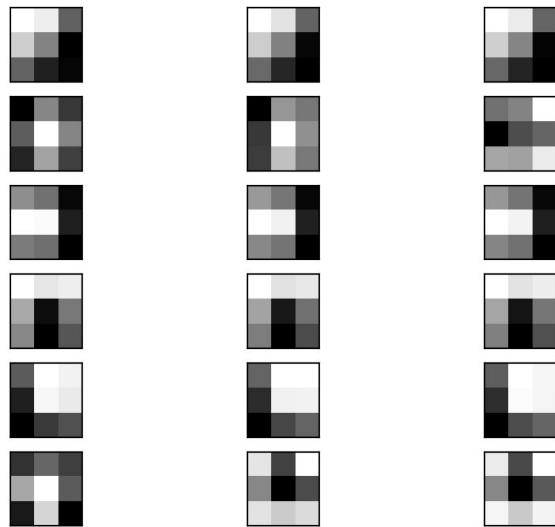
Si bien en muchos casos tiene subjetividad en la interpretación, pueden sernos **útiles** no sólo para diagnosticar problemas sino para afianzar conceptualmente cómo funcionan las redes convolucionales.

Visualización de filtros convolucionales (cont.)

Por ejemplo, una serie de **filtros 3x3**(xC) puede visualizarse mostrando en colores **claros** los valores más **altos** y en colores **oscuros** los valores más **bajos**.

A la derecha vemos **6 filtros**, con una columna para cada canal.

Sin embargo, filtros **tan pequeños** no tienen formas reconocibles.



[Fuente](#)

Visualización de filtros convolucionales (cont.)

También podemos observar cómo se activan estos filtros con una **imagen específica**.

Probemos cómo se activa la **primera capa de filtros convolucionales** para una red **VGG-16** con la imagen de la derecha (para el primer canal).

La primera capa de esta red tiene **64 kernels convolucionales** y, por lo tanto, esperamos... ¿cuántas imágenes?



[Fuente](#)

Visualización de filtros convolucionales (cont.)



[Fuente](#)

Visualización de filtros convolucionales (cont.)



[Fuente](#)

Algunos filtros capturan el cuerpo del pájaro; otros se **enfocan** en el fondo; otros, en los bordes.

Esto es esperable: si todos los filtros se enfocasen sólo en una cosa, serían **redundantes**.

Visualización de filtros convolucionales (cont.)

Veamos cómo es el **código** para observar qué pasa en los filtros convolucionales para el caso del ejemplo de la clase anterior, donde creamos una red convolucional para el dataset [EMNIST Letters](#).

Veamos `td6-p23-c-visualizacion-filtros-cnn.ipynb`.

Saliency maps por retropropagación

Visualizar los filtros convolucionales no nos dice nada respecto a **qué identifica la red para clasificar una imagen.**

En su versión más **simple**, el saliency map es básicamente:

1. Hacer una pasada *forward* con un *input*.
2. Hacer *backpropagation* de la clase de salida hasta la capa input. Es decir, no sólo calcular el gradiente de los pesos sino también respecto a los inputs.
3. Mostrar el valor absoluto del gradiente de los inputs como un *heatmap*.

¿Por qué esto funciona?

Saliency maps por retropropagación (cont.)

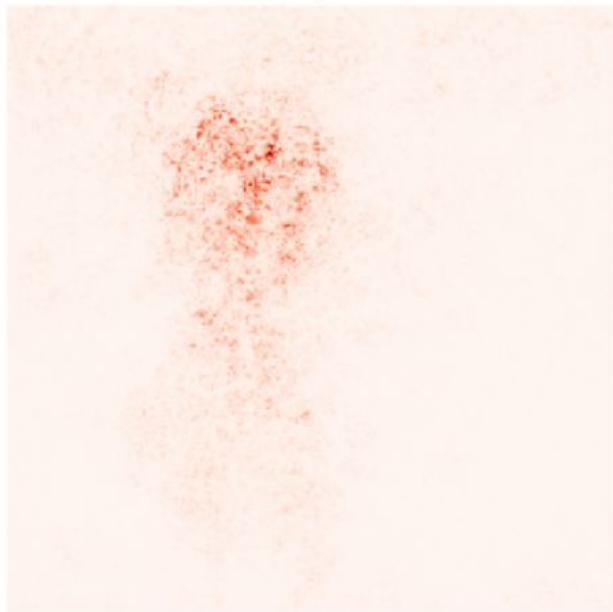
El **gradiente** nos dice cuánto cambia el valor de salida respecto al valor de alguna variable.

Entonces, podemos derivar la variable de la clase de salida (no la función de *loss*), respecto al input, y esto nos dirá **qué píxeles son importantes** (y cuánto) para la clasificación de esa clase.

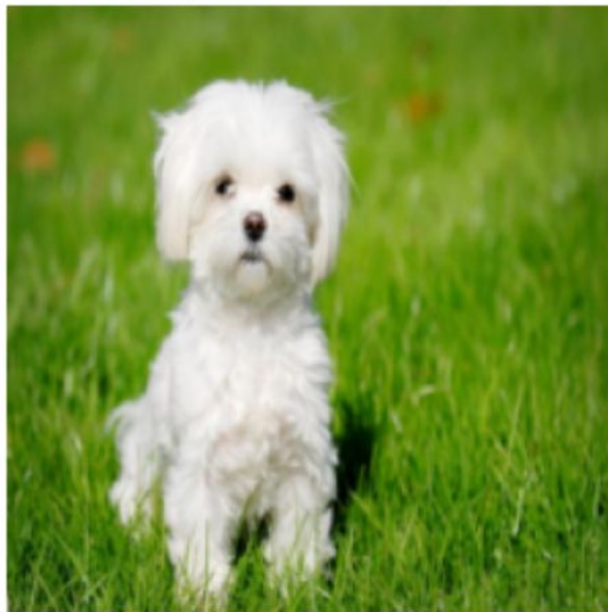
Tomamos el **valor absoluto** porque tanto los gradientes muy positivos como los muy negativos nos dicen qué píxeles son importantes para una imagen.

Saliency maps por retropropagación (cont.)

Saliency Map



Deprocessed Image



Veamos cómo **implementar** saliency maps en PyTorch. Para ello, vayamos a `td6-p22-d-saliency-maps.ipynb`.

Saliency maps por retropropagación (cont.)

Los saliency maps por retropropagación son mapas simples pero tienen algunas **desventajas**:

- Es un análisis **píxel por píxel**; no mira la imagen en su conjunto.
- Los mapas resultantes pueden ser “**ruidosos**”.
- Si bien se computa desde la salida de la clase de interés, **no es un análisis “por clase”**. Muestra todos los píxeles a los que les presta atención, pero no destaca las regiones de la imagen que tuvieron un impacto significativo al diferenciar la clase de interés.
- Es **sensible a la escala**: la magnitud del gradiente puede cambiar, si cambia la resolución de la imagen de input. El mapa de saliencia puede entonces devolver distintos resultados, dificultando su interpretación.

¿Qué otra técnica existe?

Grad-CAM

Grad-CAM (*gradient-weighted class activation mapping*) es una técnica de interpretación que se utiliza para comprender cómo un modelo de red neuronal convolucional (CNN) toma decisiones en tareas de clasificación de imágenes. **Resalta** las **regiones** específicas de una imagen que contribuyen de manera más **significativa** a la **predicción de una clase en particular**.

Referencia: [**paper de Grad-CAM**](#).

Veamos **cómo funciona**.

Grad-CAM

- 1) Obtener **Activaciones de la Última Capa Convolutiva**: Las activaciones en esta capa contienen información espacial y de características de la imagen que son importantes para la predicción.
- 2) Calcular **Gradientes de la Clase Predicha**: backpropagation para calcular los gradientes de la predicción (por ejemplo, la clase de mayor puntaje) respecto a estas activaciones.

Grad-CAM

- 3) Ponderar las Activaciones: Usa los gradientes obtenidos para calcular una **"importancia" o "peso"** para cada filtro en la capa convolucional. Esto ayuda a determinar cuánto influye cada filtro en la predicción de la clase.
- 4) Generar el **Mapa de Calor**: Se combinan las activaciones de los filtros, ponderadas por sus gradientes, para crear un mapa de calor que resalta las regiones de la imagen más relevantes para la clase predicha.
- 5) Superponer en la Imagen Original: este mapa de calor se **escala y superpone** en la imagen original,

Grad-CAM

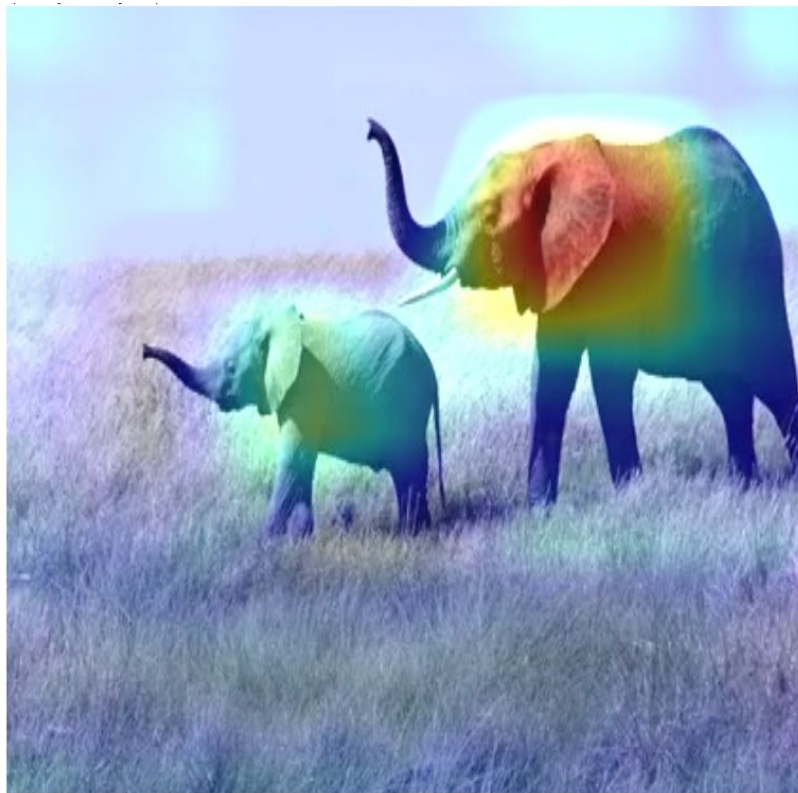
Grad-CAM es **menos sensible a la escala**, porque hace un *global average pooling* (GAP) para computar los pesos de cada feature map y, luego, hace una **suma ponderada** de los mismos. Esta suma ponderada combina información de diferentes escalas, mitigando la sensibilidad a cambios en la resolución de la imagen.

Además, esta ponderación es **específica a la clase**.

Grad-CAM

Veamos cómo podemos implementar Grad-CAM en PyTorch. Para ello, vayamos a **`td6-p22-e-grad-cam.ipynb`**.

Grad-CAM



Variedad de técnicas de interpretación

Las **técnicas** que vimos en esta clase no son las únicas. Entre las clases teóricas y prácticas, **mencionamos**:

- Mapas de saliencia por oclusión.
- Mapas de saliencia por retropropagación.
- CAM.
- Grad-CAM.

Hay **muchas formas** de interpretar los resultados de una red neuronal y el método que **elijamos depende** de la arquitectura, de las necesidades de nuestro proyecto y de las ventajas y desventajas de cada uno.

Cierre

Hoy vimos y practicamos, entre otras cuestiones,

- cómo visualizar lo que pasa dentro de los **filtros convolucionales**,
- cómo identificar a qué prestan atención las CNNs mediante **saliency maps** y
- cómo usar una alternativa más moderna de visualización llamada **Grad-CAM**.

Pueden darnos *feedback* de la clase [acá](#).